

HyDE

(Hypothetical Document
Embedding)

1. HyDE 구현

HyDE(Hypothetical Document Embeddings)는 "질문으로 검색하는 대신, 상상하여 작성한 가상의 답변으로 검색하자"는 아이디어에서 출발한 정보 검색 및 RAG(검색 증강 생성) 최적화 기법입니다.

기존 텍스트 기반 검색이나 기본 vector 임베딩 검색이 가진 대표적인 한계(질문과 정답 문서 간의 표현 차이)를 극복하기 위해 제안되었습니다.

HyDE (Hypothetical Document Embeddings)

1. HyDE란?

HyDE(Hypothetical Document Embeddings)는 검색(Query) 자체를 임베딩하지 않고, LLM이 먼저 "정답일 것 같은 가상의 문서(Hypothetical Document)"를 생성한 후, 그 문서를 임베딩하여 검색하는 기법이다.

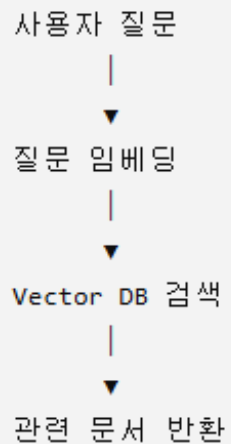
즉,

질문 → 가상의 답변 생성 → 답변 임베딩 → Vector DB 검색

이라는 과정을 수행한다.

기존 RAG는 질문 문장을 그대로 임베딩하지만, HyDE는 질문보다 훨씬 풍부한 의미를 가진 문서를 임베딩 하므로 검색 성능이 향상되는 경우가 많다.

2. 기존 RAG 검색 방식



예)

질문

LangChain Agent란?

임베딩되는 문장은

LangChain Agent란?

뿐이다.

짧은 질문이기 때문에 의미 정보가 부족할 수 있다.

3. HyDE 검색 방식

사용자 질문



LLM

(가상의 정답 문서 생성)



가상 문서 임베딩



Vector DB 검색



관련 문서 검색

예)

질문

LangChain Agent란?

LLM이 먼저 생성

LangChain Agent는 LLM이 여러 Tool을 선택하여
필요한 작업을 수행하도록 하는 프레임워크이다.
Agent는 Tool Calling과 Planning 기능을 이용하며 ...

이 긴 문서를 임베딩하여 검색한다.

4. 왜 성능이 좋아질까?

질문은 일반적으로 매우 짧다.

예)

RAG란?

FAISS

HyDE

하지만 Vector DB 안에는

RAG(Retrieval-Augmented Generation)는
LLM의 환각(Hallucination)을 줄이기 위해
외부 문서를 검색하여...

처럼 긴 문서들이 저장되어 있다.

즉

질문 벡터

↓

문서 벡터

의 형태가 크게 다르다.

HyDE는

질문

↓

가상의 문서

↓

문서 형태 벡터

를 생성하므로

문서와 벡터 공간이 훨씬 가까워진다.

5. 기존 검색 vs HyDE

기존 검색

HyDE

질문 임베딩

가상 문서 임베딩

짧은 Query

긴 Document

의미 정보 적음

의미 정보 많음

검색 실패 가능

검색 성공률 증가

빠름

조금 느림

6. 동작 과정

사용자 질문



LLM



가상의 정답 생성



Embedding



Vector Search



Top-K 문서 검색



최종 답변 생성

7. 예제

사용자 질문

HyDE가 뭐야?

LLM 생성

HyDE(Hypothetical Document Embeddings)는
질문을 직접 임베딩하지 않고,
LLM이 먼저 질문에 대한 가상의 답변을 생성한다.
생성된 답변을 임베딩하여
Vector Database에서 의미적으로
유사한 문서를 검색하는 기술이다.

이 문서를 임베딩한다.

8. LangChain 코드 흐름

Python

question



ChatOpenAI



hypothetical_document



Embedding



Retriever



관련 문서



LLM

실제로는 다음과 같은 순서가 된다.

Python

question

```
hypothetical_doc = llm.invoke(question)
```

```
vector = embedding.embed_query(hypothetical_doc)
```

```
docs = vectorstore.similarity_search_by_vector(vector)
```

9. 언제 효과적인가?

다음과 같은 경우 성능 향상이 크다.

① 질문이 매우 짧은 경우

RAG

Transformer

CUDA

② 전문 용어가 많은 경우

LoRA

QLoRA

DPO

③ 사용자가 질문을 잘 못하는 경우

LLM이 문서를 자동으로 보완

④ 문서가 긴 설명 위주인 경우

논문

매뉴얼

PDF

기술 문서

법률 문서

10. 단점

① 검색 시간이 증가

LLM 호출이 한 번 더 필요하다.

질문

↓

LLM

↓

검색

따라서 일반 검색보다 느리다.

② 비용 증가

OpenAI API를 한 번 더 호출한다.

질문 생성 비용

+

최종 답변 비용

③ 잘못된 가상 문서 생성 가능

LLM이 엉뚱한 내용을 생성하면

↓

엉뚱한 검색이 수행될 수 있다.

④ 실시간 서비스에서는 부담

대규모 서비스에서는

응답시간(latency)이 증가한다.

11. 적용 사례

- 사내 문서 검색
- 기술 매뉴얼 검색
- 논문 검색
- 법률 문서 검색
- 의료 문헌 검색
- 특허 검색
- FAQ 검색
- 기업 내부 RAG

12. 기존 Retriever와 비교

방법	특징	LLM 사용	검색 정확도	속도
Similarity Search	질문 그대로 검색	X	보통	매우 빠름
MMR Search	다양한 문서 검색	X	높음	빠름
Multi Query Retriever	여러 질문 생성	O	높음	느림
Self Query Retriever	메타데이터까지 활용	O	높음	느림
HyDE	가상 문서 생성 후 검색	O	매우 높음	느림

13. 실제 활용 예

사용자 질문

RAG에서 환각을 줄이는 방법은?

기존 검색

질문 그대로 검색



관련 문서 2개 검색

HyDE

LLM 생성

RAG에서는 Vector Database 검색,
Reranking, Chunking 전략,
Hybrid Search 등을 이용하여
환각을 줄일 수 있다...



이 문서를 임베딩



관련 문서 8개 검색



정답률 향상

14. 장단점 요약

장점

- 질문의 의미를 풍부하게 확장하여 검색 정확도를 높일 수 있다.
- 짧거나 모호한 질문에서도 관련 문서를 잘 찾는다.
- 전문 문서, 논문, 기술 문서 검색에 특히 효과적이다.
- 기존 Vector DB를 변경하지 않고 적용 가능하다.

단점

- LLM 호출이 추가되어 응답 속도가 느려진다.
- API 호출 비용이 증가한다.
- 가상 문서의 품질에 따라 검색 성능이 영향을 받는다.
- 실시간 대규모 서비스에서는 지연(latency)을 고려해야 한다.

LangChain에서는 HyDEPipeline이나 Custom Chain을 통해 쉽게 적용할 수 있다.

Python



```
from langchain.chain import HypotheticalDocumentEmbedder
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAI, OpenAIEmbeddings

# 1. 기본 임베딩 모델 및 LLM 설정
base_embeddings = OpenAIEmbeddings()
llm = OpenAI(temperature=0)

# 2. HyDE 전용 임베딩 생성기 구성
# LLM이 미리 정의된 프롬프트(web_search, scifact, precision 등)를 기반으로 가상 문서를 생성
embeddings = HypotheticalDocumentEmbedder.from_llm(
    llm,
    base_embeddings,
    custom_prompt=None # 필요시 도메인 특화 가상 문서 생성 프롬프트 지정 가능
)

# 3. Vector Store 연결 및 검색 수행
vectorstore = FAISS.from_texts(texts, embeddings)
retriever = vectorstore.as_retriever()

# 4. 검색 질문 실행 (내부적으로 LLM 가상문서 생성 -> 임베딩 -> 유사도 검색 진행)
results = retriever.invoke("HyDE의 작동 원리를 설명해줘")
```



감사합니다